

From Fragments to Evidence: Designing an Agent for the Structured Evaluation of Algorithmic Trading Strategies from QuantConnect Code

Alejandro Reynoso

Abstract

This paper presents a formal framework for building an evaluation agent capable of analyzing an algorithmic trading strategy starting from partial structural clues embedded in QuantConnect code. The central challenge is that many submitted strategies are not immediately available as clean, portable, fully specified research objects. Instead, they often arrive as platform-dependent code fragments, incomplete trading rules, ambiguous indicator references, framework-specific data assumptions, or operational placeholders. The proposed agent is therefore not merely a backtesting engine and not merely a code translator. It is a structured inference system that performs a staged investigation. It begins by extracting strategic clues from the source code, proceeds through a disciplined conversion process, reconstructs a candidate strategy representation, and then submits that representation to a controlled laboratory environment in which market-specific idiosyncrasies are stripped away and robustness can be assessed under synthetic yet governed conditions.

The paper argues that this process should be understood as a form of treasure hunting under governance. The agent first searches for the actual economic and decision logic hidden within the code. Once a candidate logic has been identified, it is not trusted immediately. Instead, it is isolated from platform dependencies, expressed in a normalized strategy form, and tested in a laboratory designed to distinguish between accidental behavior and genuine structural robustness. The resulting framework integrates code interpretation, semantic diagnostics, conversion control, synthetic scenario generation, liveness testing, regime sensitivity analysis, stress evaluation, and committee-style reporting. The purpose is not to certify profitability, but to create a reproducible, auditable, and institutionally defensible process for deciding whether a submitted strategy contains meaningful strategic content and whether that content survives disciplined scrutiny. The paper concludes that the agent should be designed as a governed evaluator of strategic structure rather than as a naive performance machine, and that such an architecture is essential for financial professionals seeking reliable AI-assisted strategy review.

Implementation Note. The reader should consult the actual implementation of the agent in the accompanying notebook, available at:

https://github.com/alexdiabol/algo_course_jbs_2026/blob/main/notebooks/QUANTCONNECT_GOVERNANCE_AGENT.ipynb

1 Introduction

The increasing use of external platforms, modular research environments, and language-model-assisted tooling has changed the practical reality of algorithmic trading research. Strategies are no longer always born and tested inside one unified stack. They may be authored in one system, reviewed in another, translated into a third, and presented to governance functions in a form that is only loosely connected to the code from which they originated. This creates a serious institutional problem. A strategy may look operational when viewed in its native framework, yet become opaque once removed from that context. Conversely, code may appear sophisticated while containing very little actual decision logic. Between these two extremes lies a large and practically important class of cases in which the evaluator must determine whether a real strategy exists, what it appears to be doing, and whether its apparent logic can withstand structured analysis.

The problem becomes especially acute when the source object is QuantConnect code. QuantConnect provides a powerful and convenient environment for strategy development, but its strategies are written within a platform-specific lifecycle. Methods such as initialization, data handlers, order functions, and indicator declarations sit inside a framework that implicitly provides objects, timing conventions, state transitions, and market data semantics. For an external evaluator, the code is therefore not a self-sufficient research object. It is a partial description of a process embedded in an external machinery. A direct reading of such code may reveal symbols, lookback windows, indicator names, execution methods, and portfolio management hints, yet still fail to disclose the true economic structure of the strategy in a portable form.

This paper proposes that the correct response is not to treat the source code as directly backtestable, nor to ask a language model for a superficial summary, but to build an agent whose task is structured strategy discovery and controlled evaluation. The agent should behave like an institutional investigator. It should begin by examining the submitted code as though it were a site containing clues. Certain references may indicate a trend-following design, others a mean-reversion architecture, others a volatility-sensitive regime rule. Yet the presence of indicators alone is not enough. The agent must ask whether the clues amount to a coherent hypothesis, whether there is actual order logic, whether risk controls are present, and whether the observed structure is sufficient to reconstruct a candidate strategy. Only once the strategy has been reconstructed should it be brought into a laboratory environment.

The laboratory is not intended to prove real-world profitability. Its role is different and conceptually more modest. It strips away the idiosyncrasies of historical data, platform machinery, and accidental implementation details, and then examines whether the candidate strategy retains any coherent structure under controlled perturbation. In this way the process resembles a filtering mechanism. One begins with a possibly messy artifact from a production-oriented platform, extracts a candidate strategic core, removes environment-specific noise, and then asks whether anything robust remains. If the answer is yes, the strategy may deserve deeper review. If the answer is no, the agent should say so clearly and with evidence.

The architecture proposed here is therefore fundamentally governance-first. It is not designed to maximize optimism, and it is not designed to overstate the significance of synthetic backtests. Instead, it is designed to answer a more basic and more defensible question: does

the submitted object contain a strategy-like structure that can survive disciplined translation and controlled testing? This question is deeply relevant to financial professionals. Investment committees, model reviewers, quantitative researchers, and risk managers all need processes that distinguish genuine strategic structure from noise, ambiguity, and inert code. A reliable evaluation agent should not merely compute returns. It should identify semantic insufficiency, detect silent failure, document uncertainty, and package the resulting evidence in a form suitable for institutional review.

This paper develops such an agentic framework in a formal and professional manner. It begins by defining the nature of the source problem, then describes the treasure-hunting stage in which the agent extracts strategic clues from QuantConnect code. It next explains the laboratory stage in which strategy hypotheses are normalized and subjected to synthetic robustness analysis. Finally, it presents a governance architecture that ensures the agent's outputs remain auditable, constrained, and decision-useful. The result is a practical blueprint for building an agent that can transform platform-dependent code into a defensible object of analysis.

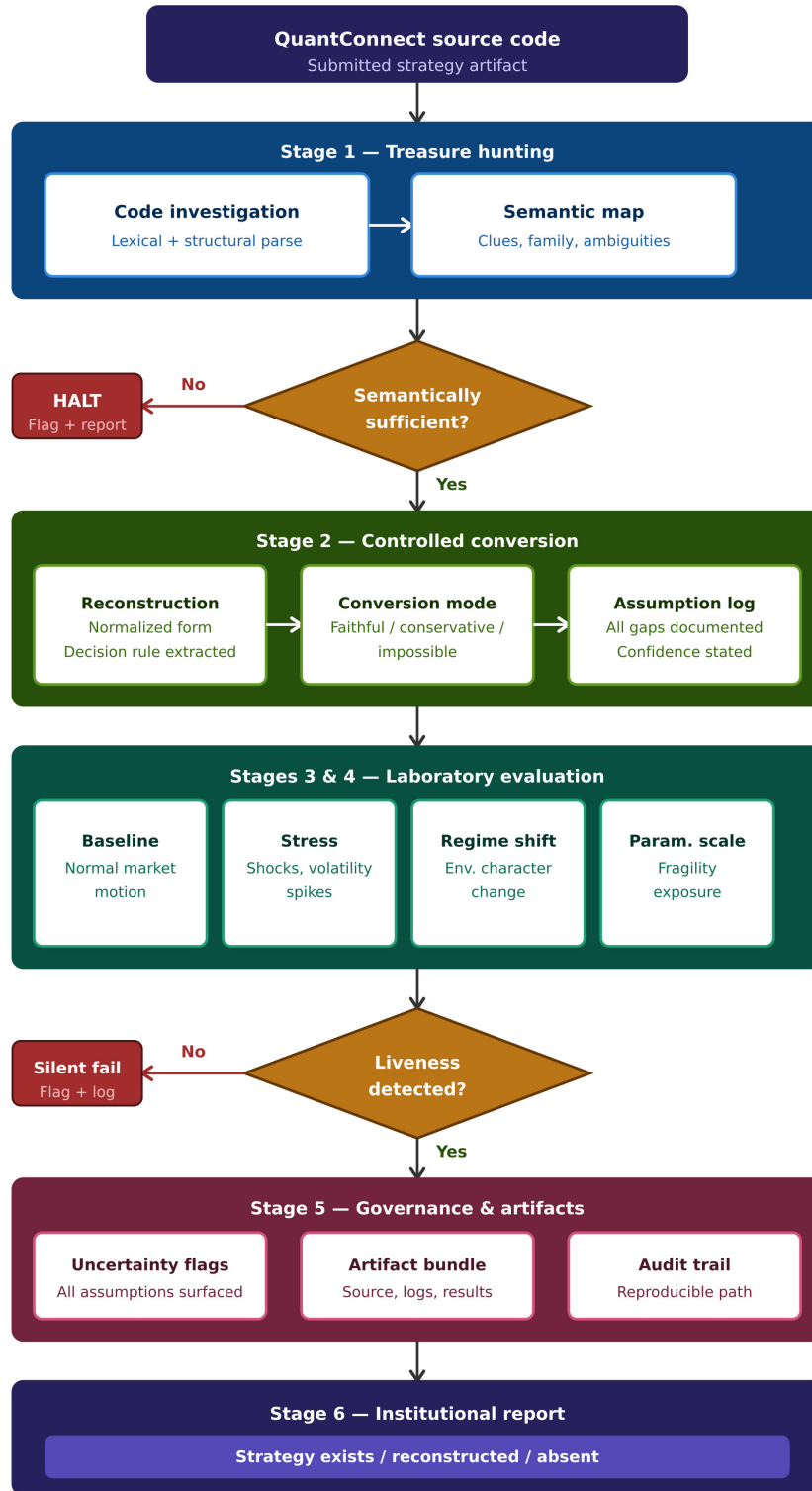


Figure 1: Strategy Evaluation Flowchart

2 The Source Problem: Strategies as Incomplete Objects

A submitted trading strategy is rarely a complete economic proposition in its raw coding form. In principle, one might imagine that source code fully specifies the strategy, but in practice this is often false. A piece of QuantConnect code may contain declarations of symbols, indicators, and parameter values while omitting the actual causal chain that converts information into trades. A method may exist but contain no execution logic. Risk management may be implicit, distributed across multiple helper functions, or absent altogether. The code may run inside its native environment precisely because the platform supplies contextual assumptions that are invisible to an outside evaluator.

For this reason, the first conceptual move in designing an evaluation agent is to reject the naive assumption that code is self-explanatory. The agent must instead treat the code as an incomplete object containing several layers of information. The first layer is lexical: variable names, method calls, and platform functions. The second is structural: the flow from initialization to data updates to execution. The third is economic: the latent hypothesis about market behavior implied by the chosen indicators, assets, and timing rules. The fourth is operational: the presence or absence of actual portfolio actions, order management, and exposure control. The agent’s job is to move from the lexical and structural layers toward the economic and operational layers without fabricating what is not present.

This is why the treasure-hunting metaphor is useful. The evaluator begins with fragments. A lookback window suggests memory. A moving average suggests smoothing. A volatility reference suggests sensitivity to uncertainty. A benchmark symbol may imply comparison or hedging. A call to liquidate may imply a reset rule. These are clues, not conclusions. A professional agent must accumulate them, weigh them, and determine whether they converge into a plausible strategic identity. If they do not, the agent must say that the source is semantically incomplete. Such an outcome is not a failure of the evaluator. It is a valid finding.

From a governance perspective, the source problem can be stated more sharply. The objective is not merely to infer a strategy, but to classify the source according to convertibility. Some strategies are fully actionable and can be ported with high confidence. Others can be conservatively reconstructed with substantial caveats. Others permit no faithful conversion at all. This taxonomy matters because it prevents the evaluation system from overclaiming. If the agent cannot recover a meaningful decision rule, it should not proceed as though it had done so. Instead, it should preserve the distinction between a functioning strategy, a hypothesized reconstruction, and an empty shell.

3 Stage One: Treasure Hunting in Code

The first operational stage of the agent is code investigation. In this stage the source is parsed not simply as software, but as evidence. The agent should examine the QuantConnect submission for specific classes of clues. These include the declared universe of tradable assets, indicator choices, lookback horizons, entry and exit methods, order instructions, portfolio sizing references, liquidation behavior, benchmark comparisons, and any explicit thresholds

or state transitions.

The most important principle at this stage is that clue extraction must precede conclusion. An indicator by itself is not a strategy. A symbol by itself is not an economic thesis. Even a sequence of order-related calls may fail to establish a coherent rule if the trigger conditions are unclear. The agent must therefore construct an intermediate representation that distinguishes between observed facts and inferred hypotheses. This is the beginning of disciplined interpretation.

A robust implementation should produce a diagnostic inventory of the source code. The inventory should identify whether initialization is present, whether data-handling methods exist, whether order instructions appear, whether indicators are declared, whether the code uses helper functions, and whether essential methods are operational or empty. In particular, the agent should identify liveness clues. If the primary data method is present but does nothing, or if no order-generating logic exists, the agent should classify the source as semantically insufficient. This is a crucial control. It prevents the downstream pipeline from mistaking absence of behavior for conservative strategy design.

The treasure-hunting stage should also identify candidate strategic families. A trend strategy typically leaves a different clue pattern than a mean-reversion strategy. A breakout design differs from a carry design; a cross-sectional rotation rule differs from a single-asset timing rule. These families matter because they determine what kinds of normalized strategy representations may later be constructed. Yet the agent must remain conservative. Family classification should not be treated as proof, but as a working hypothesis to be validated or rejected later.

An effective agent therefore produces more than a code summary. It produces a semantic map. The map should contain detected features, unresolved ambiguities, missing components, and a provisional classification of the strategy type. This map becomes the handoff document between the treasure-hunting stage and the conversion stage.

4 Stage Two: Controlled Conversion and Strategy Reconstruction

Once the agent has extracted clues and formed a semantic map, it must perform controlled conversion. This stage is not equivalent to ordinary code transpilation. The goal is not merely to rewrite syntax. The goal is to reconstruct a portable and evaluable representation of the candidate strategy while preserving explicit uncertainty about what is known and what is not.

The converted object should take a normalized form. At a minimum, it should contain a feature-preparation component, a decision component, configuration metadata, and explanatory annotations. This normalized representation serves several purposes. First, it isolates the strategy from its original platform dependencies. Second, it creates a stable interface for the downstream laboratory. Third, it forces the conversion process to state its assumptions.

The discipline of this stage lies in explicit conversion modes. If the source provides a clear and operative decision structure, the conversion may be classified as faithful. If the source is incomplete but still allows a cautious reconstruction, the conversion should be classified

as conservative reconstruction. If the source fails to provide enough semantic content for a plausible decision rule, the conversion should be classified as impossible faithful conversion. This final category is essential. Without it, the agent will tend to produce superficially plausible but strategically empty artifacts.

In institutional terms, the conversion stage is a model risk stage. Every assumption introduced during reconstruction should be documented. Unused symbols, ambiguous thresholds, missing risk controls, and inferred execution timing should all be surfaced. Confidence should be reported explicitly. The downstream laboratory must know whether it is testing a faithful strategy or a speculative approximation.

This stage also clarifies a deeper conceptual point. A strategy is not simply a collection of indicators. It is a mapping from observable states to portfolio actions. The conversion process must therefore seek the action rule. If the rule cannot be found, the agent should not fabricate one merely to satisfy the later stages of the pipeline. To do so would destroy the integrity of the evaluation.

5 Stage Three: Submission to the Laboratory

Once a candidate strategy representation exists, it should not yet be judged in the messy historical context from which it emerged. Instead, it should be submitted to a laboratory. The purpose of the laboratory is to strip away source-specific idiosyncrasies and expose the underlying structure of the candidate strategy to controlled variation.

The laboratory should be understood as a normalized evaluation environment. Rather than asking whether the strategy “worked” in the original data, the laboratory asks whether the inferred decision structure behaves coherently under stylized conditions. This is where the filtering metaphor becomes central. The source code arrives entangled with framework behavior, historical accidents, and implementation quirks. The laboratory removes these entanglements by re-expressing the strategy in standardized conditions and then observing what remains.

A well-designed laboratory includes several scenario classes. A baseline scenario provides ordinary market motion without extreme dislocation. A stress scenario introduces shocks, elevated volatility, or adverse trend changes. A regime-shift scenario tests whether the strategy adapts, breaks, or remains inert when the statistical character of the environment changes. Parameter-scaled scenarios test whether the strategy’s behavior is tied to arbitrary price levels rather than structural conditions. The exact construction of these scenarios may vary, but the principle is stable: the laboratory examines structural robustness, not market prophecy.

This stage is especially valuable because it can reveal silent failure. A strategy that appears syntactically correct may show zero trades, zero exposure, or identical outcomes across radically different environments. Such results are highly informative. They suggest that either no strategy exists, or the reconstructed decision rule has no operational content. In both cases, the laboratory has done its job. It has filtered out the illusion of complexity and exposed the absence of actual strategic behavior.

6 Stage Four: Stripping Idiosyncrasy and Testing Robustness

The central intellectual objective of the laboratory is to strip away idiosyncrasy. Historical market data contains specific paths, specific accidents, and specific coincidences. Platform-dependent code contains framework quirks, helper abstractions, and latent assumptions. If one wants to assess the robustness of the strategy as a structure, one must detach it from these contingent features.

This does not mean eliminating realism in a simplistic way. Rather, it means isolating what is essential. If a candidate strategy depends entirely on one accidental historical drift path, it should weaken once exposed to alternative synthetic regimes. If it depends on one arbitrary price scale, scaled scenarios should expose that fragility. If it is genuinely structured around some broader market regularity, its behavioral identity should remain at least partially intelligible across controlled perturbations.

Robustness here should be defined carefully. It does not mean profitability in every scenario. Nor does it mean stability of returns under all perturbations. A strategy may be structurally robust in the sense that it continues to behave according to its inferred thesis, even while losing money in an unfavorable environment. The agent should therefore focus on behavioral coherence, liveness, sensitivity patterns, and internal consistency. These are more defensible early-stage criteria than raw synthetic performance.

The filtering process can thus be interpreted as a sequence of questions. Does the strategy act at all? Does it act differently when the environment changes? Are those differences intelligible in light of the hypothesized strategy family? Does the reconstructed rule exhibit fragile dependence on accidental features? Does the source contain enough content to justify any conclusion at all? These questions are more valuable for institutional screening than the simplistic question of whether the synthetic Sharpe ratio appears attractive.

7 Stage Five: Governance, Liveness, and Fail-Closed Design

No evaluation agent for trading strategies should be considered complete without a governance architecture. The danger of strategy review is not only false optimism, but false procedural confidence. A pipeline that runs smoothly while evaluating semantically empty code is a governance failure even if every file is neatly written.

For this reason, the agent should include fail-closed mechanisms at several stages. At the source stage, semantically empty code should trigger a convertibility warning or outright halt. At the conversion stage, low-confidence reconstruction should be flagged explicitly. At the laboratory stage, zero-trade and zero-exposure behavior should trigger liveness alerts. At the reporting stage, the final memorandum should distinguish between a poor strategy and the absence of a recoverable strategy.

Liveness deserves special emphasis. In many technical systems, inactivity is difficult to distinguish from conservative prudence. A strategy that never trades may appear safe, but in fact it may simply be inert. The agent should therefore treat persistent zero exposure

and zero trade counts as potential failure signals requiring explicit classification. This is not a cosmetic reporting choice. It is essential to avoid silent non-strategies being passed downstream as if they had survived evaluation.

The governance layer should also preserve artifacts. The submitted source, extracted clues, conversion metadata, laboratory scenarios, results, flags, and final reports should all be logged and bundled. The agent’s output should not be a single numerical verdict. It should be an audit pack. This pack is what allows a human reviewer to inspect the path from source to conclusion.

8 Stage Six: The Reporting Agent and Institutional Output

The final stage of the process is report synthesis. Here the agent transforms the technical evidence into a form suitable for committee review. A robust report should include the classification of the source, the conversion mode, the key uncertainties, the observed laboratory behavior, the major governance flags, and the recommended next actions.

Crucially, the reporting agent should be trained to maintain epistemic discipline. It must not confuse synthetic robustness analysis with live-market validation. It must not turn low-confidence reconstruction into exaggerated strategic interpretation. It must not frame inactivity as defensive excellence. It must remain precise, cautious, and institutionally useful.

This is where the entire architecture comes together. The report is not a mere summary. It is the formal output of a structured inference process. It tells the committee whether a strategy appears to exist, whether it has been reconstructed faithfully or only approximately, whether it behaves coherently under controlled filtering, and whether the object deserves escalation, redesign, or rejection.

In many organizations, this reporting stage is what turns a technical notebook into an operational evaluation system. Senior decision-makers often do not want raw logs, but they do want a clear statement of what was learned and what remains unknown. The reporting agent therefore serves as the bridge between machine evidence and institutional judgment.

9 Conclusion

The problem of evaluating algorithmic strategies from QuantConnect code is not fundamentally a problem of code execution. It is a problem of structured discovery under uncertainty. The evaluator does not begin with a perfectly specified economic object. It begins with fragments, hints, platform assumptions, and variable levels of semantic completeness. The correct response is therefore not to force an immediate backtest, but to build an agent that behaves as a disciplined investigator.

Such an agent begins by treasure hunting. It scans the source code for clues about the strategy’s latent structure, identifies whether an actionable decision rule exists, and distinguishes between observed facts and inferred hypotheses. It then performs controlled conversion, not as naive syntax rewriting but as governed strategy reconstruction with explicit confidence and clearly stated assumptions. Once a candidate strategy has been formed,

it is submitted to a laboratory that strips away historical and platform idiosyncrasies, subjects the strategy to controlled scenario analysis, and asks whether any coherent structural behavior survives. Throughout the process, the agent records uncertainty, detects liveness failures, preserves artifacts, and ultimately produces an institutional report rather than a casual commentary.

The core achievement of this architecture is that it separates three cases that are too often conflated: a genuinely operating strategy, a conservatively reconstructed strategy, and the absence of a recoverable strategy. This separation is indispensable for financial professionals. Without it, evaluation pipelines become dangerous precisely when they appear cleanest. A semantically empty submission may glide through the system, receive charts, acquire a report, and masquerade as a reviewed model. The governed agent described in this paper is designed to prevent that failure.

At a broader level, the framework offers a model for how AI-assisted financial analysis should be built. The language model is not asked to pronounce truth. It is placed within a constrained institutional architecture where its roles are bounded, its assumptions are exposed, and its outputs are logged. The laboratory does not pretend to certify investability. It asks more defensible questions about coherence, liveness, fragility, and structure. The final reporting layer does not oversell synthetic results. It translates technical evidence into reviewable judgment.

In this sense, the proposed agent is best understood not as a performance oracle, but as an instrument of disciplined strategy triage. Its purpose is to determine whether something worth calling a strategy can be found within the source, whether that object retains coherence once stripped of platform-specific noise, and whether the resulting evidence justifies deeper institutional attention. That is a narrower aim than proving alpha, but it is a more honest and more useful one. In modern quantitative finance, where code moves across environments and AI increasingly participates in research workflows, such honesty is not a luxury. It is a prerequisite for governable practice.

References

- [1] Lo, Andrew W. “The Statistics of Sharpe Ratios.” *Financial Analysts Journal*, Vol. 58, No. 4, 2002, pp. 36–52. doi:10.2469/faj.v58.n4.2453.
- [2] Bailey, David H., Jonathan M. Borwein, Marcos López de Prado, and Qiji Jim Zhu. “The Probability of Backtest Overfitting.” *Journal of Computational Finance*, Vol. 20, No. 4, 2017, pp. 39–69. doi:10.21314/JCF.2017.325.
- [3] Bailey, David H., and Marcos López de Prado. “The Deflated Sharpe Ratio: Correcting for Selection Bias, Backtest Overfitting and Non-Normality.” *The Journal of Portfolio Management*, Vol. 40, No. 5, 2014, pp. 94–107. doi:10.3905/jpm.2014.40.5.094.
- [4] López de Prado, Marcos. *Advances in Financial Machine Learning*. Hoboken, NJ: John Wiley & Sons, 2018. ISBN 978-1-119-48208-6.

- [5] Granger, Clive W. J., and Allan Timmermann. “Efficient Market Hypothesis and Forecasting.” *International Journal of Forecasting*, Vol. 20, No. 1, 2004, pp. 15–27. doi:10.1016/S0169-2070(03)00012-8.